

■ КУРС • ANDROID • KOTLIN

Введение в мобильную разработку

Архитектура Android и инструментарий

Разработка мобильных приложений на Kotlin • Проектный курс



AUTHOR · NEXIGN TEAM

■ СПИКЕР

Александр Митясов

Старший специалист по технологическому обучению

Nexign · Команда технологического обучения

СПЕЦИАЛИЗАЦИЯ

- Технологии JVM: Java / Kotlin
- Spring Framework, Groovy
- Логирование и мониторинг высоконагруженных приложений

КУРСЫ

[Java Fundamentals]

[Spring]

[Groovy]

[Observability]

■ NEXIGN

В2В-разработка ПО для телекома

Давайте познакомимся — несколько цифр о компании

СОТРУДНИКОВ

2000

по всему миру

ФИЛИАЛОВ

11

в разных странах

ПРОДУКТОВ

25+

технологических решений

ПРОЕКТОВ

150+

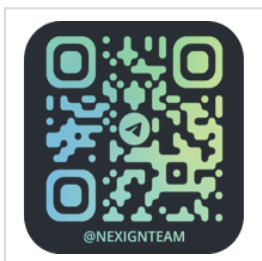
реализовано для клиентов

01 ■

TELEGRAM

Telegram-канал

Канал с новостями о компании



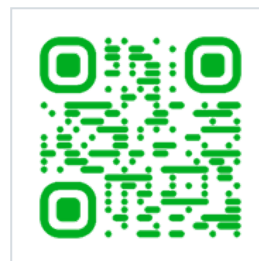
отсканируйте QR

02 ■

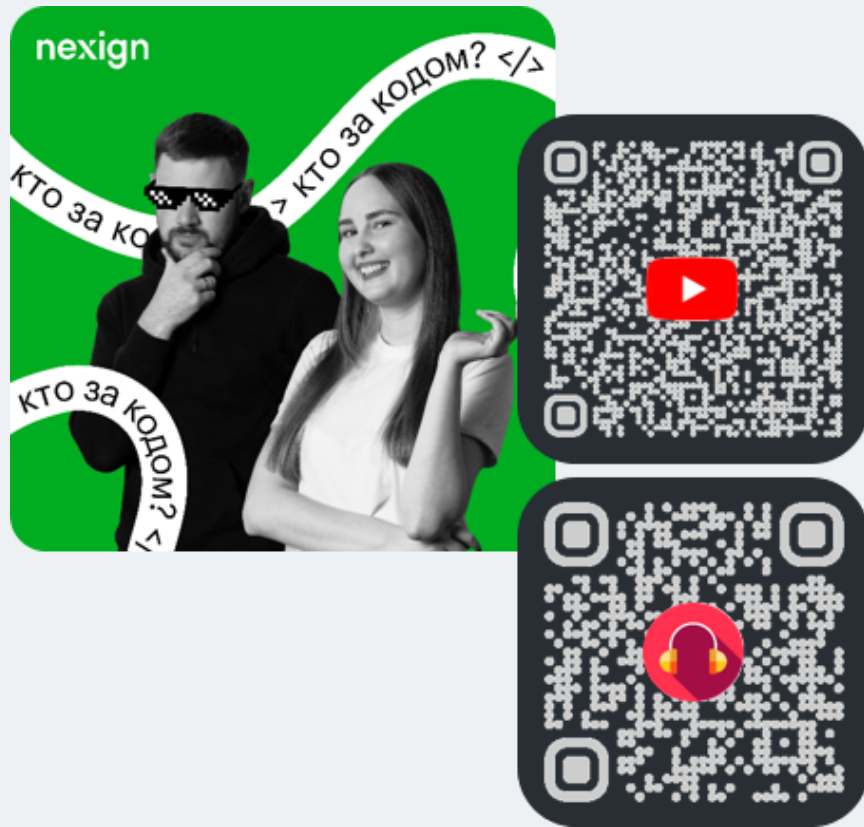
CAREERS

Карьерные возможности

Где искать вакансии и стажировки — для тех, кто захочет развиваться в В2В-разработке после курса.



nexign.com/careers



PODCAST · NEXIGN TEAM

■ PODCAST

Кто за кодом?

Подкаст про разработку, технологии и людей за кодом

- Интервью с инженерами и лидами индустрии
- Разговоры про технологии, карьеру и культуру разработки
- Выходит на площадках Nexign

[Apple Podcasts](#)[Spotify](#)[YouTube](#)[Яндекс.Музыка](#)

Курс и формат

Проектный курс разработки мобильных приложений на Kotlin

- Проектный курс: за семестр вы делаете **своё Android-приложение** — от идеи до APK.
- Цикл недели: **лекция → задание на GitHub → проверка на практике (после PR)**.
- 8 разделов → 8 заданий (0–10 б) + защита итогового проекта (до 20 б).
- Итог: до 100 б, диф.зачёт.

8

РАЗДЕЛОВ

80

БАЛЛОВ ЗА ЗАДАНИЯ

100

ИТОГ, ДИФ.ЗАЧЁТ

■ ПОЗИЦИЯ КУРСА

Отношение к AI

Использовать можно. Но **понимать** — обязательно. AI — инструмент, а не замена инженера.

■ ОСНОВНОЙ ТЕЗИС

«Не быть **meat proxy** — ретранслятором чужого кода без **понимания**.»

Meat proxy — разработчик, который копирует ответы AI, не разбираясь в коде и не умея их воспроизвести.

01

■ ПРИНЦИП 01

Понимать

Что именно делает сгенерированный код. Какие данные он обрабатывает, какие ограничения есть, какие edge cases не учтены.

02

■ ПРИНЦИП 02

Запустить руками

Собрать проект, запустить на эмуляторе, пройти по шагам. Уметь отлаживать без подсказок AI.

03

■ ПРИНЦИП 03

Объяснить

Ответить на вопрос «почему этот код?». Рассказать преподавателю и коллегам, как работает каждая строчка.

04

■ ПРИНЦИП 04

Сделать подобное

Воспроизвести решение самостоятельно — без AI под рукой. Если не получается — значит, AI пока рано доверять эту задачу.

■ ДИАГНОСТИКА

Входной тест

Точка входа — не оценка. Срез по 5 темам: git, Docker, ООП, Linux, программирование. Помогает настроить темп курса.

- 01 Результат не влияет на зачёт.
- 02 **5 тем** : git, Docker, ООП, Linux, основы программирования.
- 03 **до 20 минут** на выполнение. Не знаете — пишите "не знаю", это тоже результат.



SCAN • ENTRY TEST

Что установить дома

Подготовка окружения до первой практики

01

ANDROID STUDIO

IDE + эмулятор + SDK

Последняя стабильная версия Quail. Включает JBR — встроенную Java, отдельный JDK ставить НЕ нужно.

~8 ГБ ОЗУ min

16 ГБ лучше · ~30 ГБ диск

02

GIT

Контроль версий

Для сдачи заданий через GitHub. Команды: clone, commit, push, PR.

CLI / GUI

03

GITHUB ACCOUNT

Аккаунт + организация курса

Создать, зайти в организацию курса по инвайту из чата. Задание = pull request.

REQUIRED

ВРЕМЯ УСТАНОВКИ

Первая загрузка качает SDK ~несколько ГБ — может занять **час и больше**.
Ставить **до практики**.

ПОДДЕРЖКА

Не ставится — приносите ноутбук за 15–20 мин до практики, поможем. Подробная инструкция — `INSTALL_ANDROID_STUDIO.md` в чате.

Почему мобильная разработка

Android — первая платформа в мире по доле устройств

РЫНОК

~3.6B

пользователей мобильных устройств в мире

ДОЛЯ ANDROID

~71%

глобальная доля Android среди мобильных ОС

РЫНОК РФ

~70%+

доля Android на рынке России

ПОЧЕМУ ANDROID

- Открытость платформы — кастомные ROM, эмуляторы, ADB, AOSP
- Низкий порог входа — бесплатная Android Studio, официальные гайды
- Эмуляторы — телефон не нужен, тестируем на виртуальном устройстве
- Высокий спрос на разработчиков — marketplace и корпоративные проекты

ДЛЯ РАЗРАБОТЧИКА

- Один навык → приложение на миллионы устройств
- Открытый код Jetpack-библиотек и образцов Google
- Kotlin Multiplatform — переход на iOS и десктоп без переписывания логики

Почему Kotlin

Основной язык Android по версии Google

- 01 Kotlin-first с Google I/O 2019 — новые Android API пишутся сначала на Kotlin.
- 02 Jetpack Compose работает **только** на Kotlin.
- 03 Современный, null-безопасный, лаконичный синтаксис. ~20% меньше падений в проде (данные Google).
- 04 Java остаётся совместимой (интероп), но для **новых** проектов — Kotlin.

■ ОФИЦИАЛЬНЫЙ СТАТУС

«Kotlin — основной язык Android по версии Google.»

JetBrains, 1.0 — 15.02.2016. Имя — остров Kotlin под Петербургом.
Официальная поддержка Android с 2017.

>50%

проф. Android-разработчиков на Kotlin

>95%

топ-1000 приложений в Play содержат Kotlin

Источник: kotlinlang.org

Kotlin: главные особенности

Три вещи, которые сразу отличают Kotlin от Java

01

NULL SAFETY

Null-безопасность

В Java любая ссылка может оказаться null — `NullPointerException` валит приложение. В Kotlin тип сам говорит: `String` — никогда не null, `String?` — может. Компилятор заставляет обработать случай ещё на этапе написания.

```
// Kotlin — null в типе
var name: String = "A"
var nick: String? = null

// безопасный вызов
val len = nick?.length ?: 0
```

02

DATA CLASSES

Выразительность

Одна строка — и у вас есть `equals`, `hashCode`, `toString`, `copy`. В Java за то же самое — десятки строк boilerplate кода.

```
// Kotlin — 1 строка
data class User(
    val id: Int,
    val name: String
)

// авто-генерируется:
// equals, hashCode,
// toString, copy, componentN
```

03

LAMBDA + EXT

Функции как значения

Лямбды, функции-расширения, функции высшего порядка. Кода меньше, смысл тот же. Kotlin на 100% совместим с Java — вызываем Java-код из Kotlin и наоборот.

```
// функция-расширение
fun Int.square() = this * this

// лямбда как параметр
nums.filter { it > 0 }
    .map { it.square() }
```

СОВМЕСТИМОСТЬ Kotlin на 100% совместим с Java — вызываем Java-код из Kotlin и наоборот. Детально разберём в Разделах 2–3.



Kotlin и Java: JVM, JDK

Где живут Kotlin и Java, и что особенного в Android

- JVM** (Java Virtual Machine) — виртуальная машина, исполняет байткод на любом компьютере. Принцип «написал один раз — запустил где угодно».
- JDK** (Java Development Kit) — набор: компилятор (`javac` для Java, `kotlinc` для Kotlin) + JVM + стандартные библиотеки.
- И Kotlin, и Java компилируются в **байткод JVM** — отсюда общая экосистема библиотек и инструментов.
- В Android на устройстве код исполняет **не JVM**, а **ART** — Android Runtime. Байткод — не `.class`, а **DEX** (`classes.dex` внутри APK).

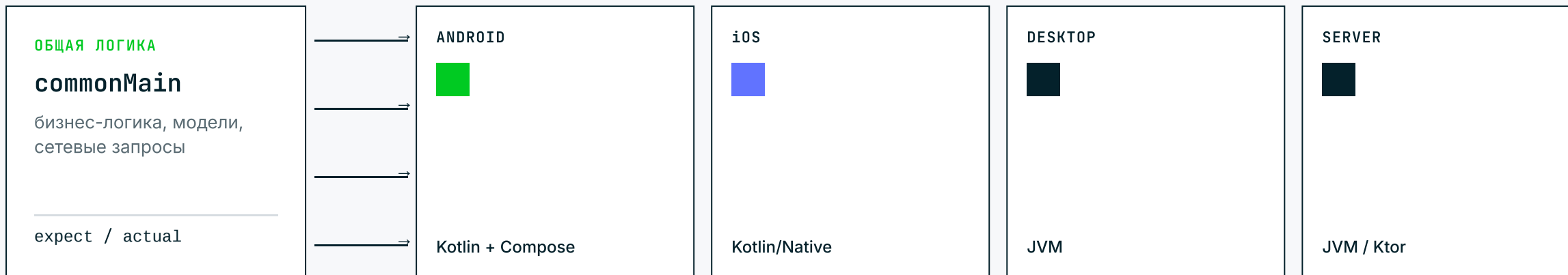
ВАЖНО ДЛЯ ANDROID

Для работы со студентами **отдельно ставить JDK не нужно** — Android Studio приносит свою версию Java (JBR — JetBrains Runtime) внутри.

`kotlinc` → JVM-байткод → `d8` → DEX → ART на устройстве

Kotlin Multiplatform

Один Kotlin-код — несколько платформ. Тема для кругозора, не для курса



COMPOSE MULTIPLATFORM

Тот же UI — на всех платформах

Настройка JetBrains: API как у Jetpack Compose, но UI запускается и на iOS, и на десктопе. На Android Compose Multiplatform фактически и есть Jetpack Compose.

ПОЧЕМУ ВАЖНО

Принципы переносятся

Принципы Compose, которые вы выучите для Android, не пропадут — если захотите выйти на iOS, часть кода переиспользуется. В рамках курса KMP НЕ проходим — упоминаем как возможность роста.

Архитектура платформы Android

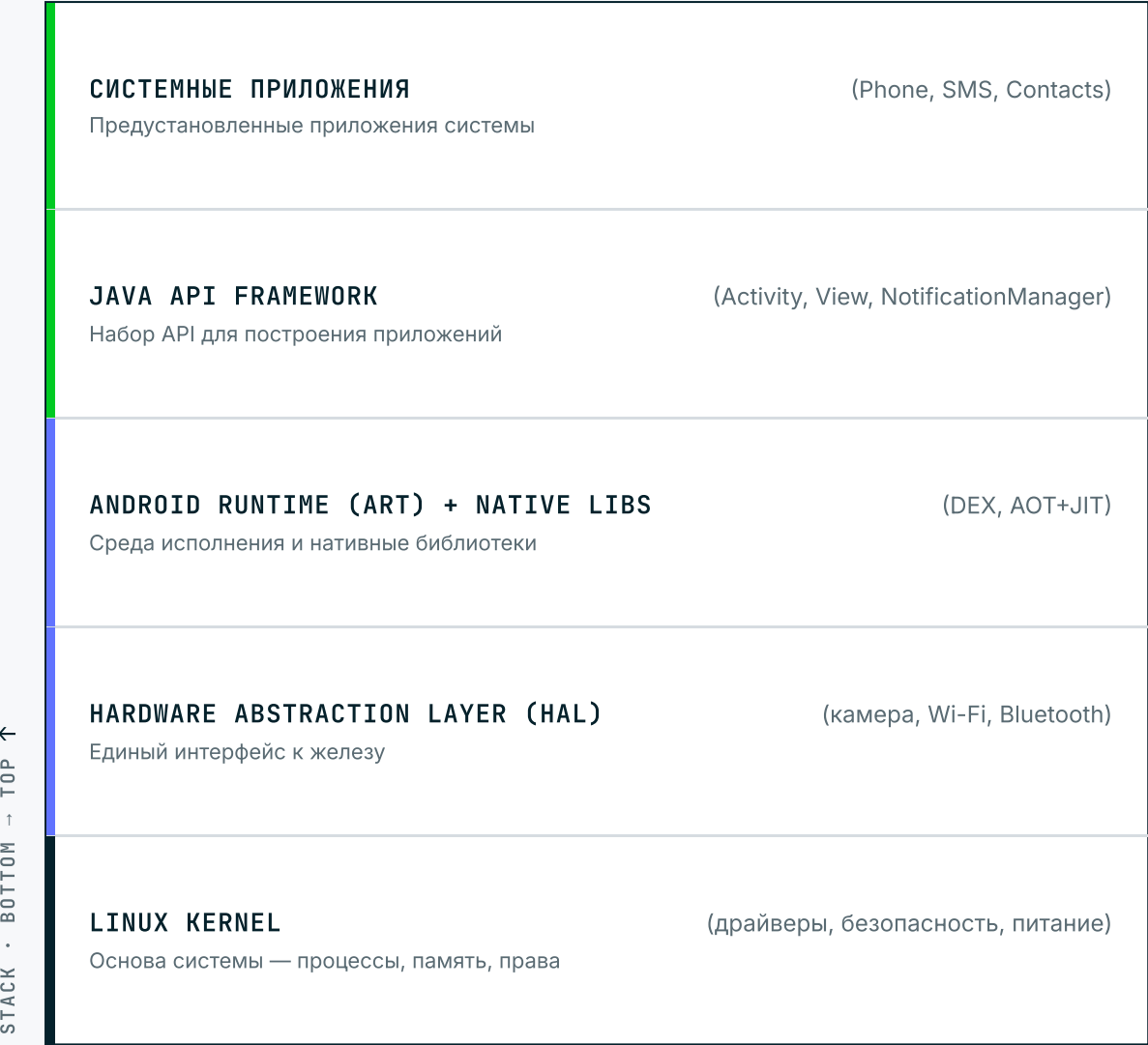
Слоёная структура — от ядра Linux до приложений

- 01 **Linux kernel** — основа: процессы, память, безопасность.
- 02 **HAL** — единый интерфейс к железу (камера, Wi-Fi, Bluetooth).
- 03 **ART** — среда исполнения (с Android 5.0; ранее — Dalvik). AOT+JIT, DEX-байткод.
- 04 **Песочница**: каждое приложение — в своём процессе. Не может читать чужую память — только через разрешения пользователя.

ИСТОРИЧЕСКАЯ СПРАВКА

Dalvik — название среды исполнения до Android 5.0 (2014). Сейчас — ART, мы уже это знаем.

Источник: developer.android.com/guide/platform



Как устроено приложение сегодня

Классические 4 компонента → современная модель single-activity + Compose

■ КЛАССИКА

4 компонента Android

Нужно уметь узнавать в чужом коде

- **Activity** — экран с UI
- **Service** — фоновая работа без UI
- **BroadcastReceiver** — реакция на события системы
- **ContentProvider** — обмен данными между приложениями

LEGACY · 4 COMPONENTS

■ СОВРЕМЕННАЯ МОДЕЛЬ

Single-activity + Compose

Рекомендация Google, наш план на курс

- Одна Activity-обёртка — контейнер для Compose-экранов
- Compose-экраны внутри — каждый со своим состоянием
- **ViewModel** переживает поворот экрана
- **Repository + Hilt + Flow** — данные и зависимости

COMPOSE-FIRST · 2026

01 LIFECYCLE

Система управляет экраном

Создался → на переднем плане → свёрнут → закрыт.
Подробно разберём в **Разделе 6** — сегодня только идея.

02 VIEWMODEL

Данные переживают поворот

Повернули телефон — Android **пересоздаёт экран**.
Данные хранят не в Activity, а в ViewModel — она переживает пересоздание.

03 STACK

Compose + ViewModel + Hilt + Flow

Именно такой стек просят в вакансиях. К нему и идём весь семестр.

■ JETPACK COMPOSE

Jetpack Compose

Декларативный UI-тулkit Google для Android

Стабилен с 28 июля 2021. С 2026 — **Compose-first** у Google: всё новое пишется на Compose, старый Views перешёл в режим поддержки.

- 01 **Декларативный:** вы описываете, как экран выглядит при текущем состоянии, а не «найди кнопку, поставь текст, повтори».
- 02 **Composable-функции** с аннотацией `@Composable` — UI собирается из вложенных функций: `Column { Row { Button(...) } }`.
- 03 **Реактивность:** данные изменились — Compose **сам перерисует** то, что зависит от них (рекомпозиция).

ФАКТ

68% топ-1000 приложений в Play используют Compose (данные Google к 5-летию, июль 2026).

■ ЧТО ЗАМЕНЯЕТ

Compose заменяет

- **XML-разметку** (`activity_main.xml` , `layout/*.xml`)
- **View-систему** (`View` , `findViewById()` , View Binding)
- **Fragments + RecyclerView** с адаптерами
- **Один экран = одна Activity** — теперь одна Activity-обёртка

■ НЕ ЗАМЕНЯЕТ

- Activity, Service — компоненты ОС
- ViewModel, Room, Retrofit
- Gradle, SDK, манифест

Compose — слой UI, а не вся ОС

Composable vs обычная функция

Ключевой слайд — вся разница в одном предложении: обычная функция вычисляет значение, composable описывает интерфейс

	ОБЫЧНАЯ ФУНКЦИЯ	COMPOSABLE-ФУНКЦИЯ (@COMPOSABLE)
Что делает	Вычисляет значение / выполняет действия	Описывает, как выглядит часть экрана
Аннотация	нет	@Composable
Откуда вызывать	откуда угодно	только из других composable
Зависит от	своих параметров	параметров + реактивного состояния (mutableStateOf)
При изменении данных	ничего (вызовете сами ещё раз)	автоматически перерисовывается (recomposition)

■ ОБЫЧНАЯ ФУНКЦИЯ

Считает и возвращает

```
// Обычная функция — считает
fun double(x: Int): Int = x * 2

// Вызов откуда угодно
val result = double(5) // 10
```

Выполнилась — забыла. Чтобы обновить результат, вызовете снова сами.

■ COMPOSABLE

Описывает интерфейс

```
// Composable — рисует UI
@Composable
fun Greeting(name: String) {
    Text(text = "Привет, $name!")
}

// Вызов только из composable
// Greeting("A") // ОК если из @Composable
// Greeting("A") // ОШИБКА если из обычной
```

Compose вызывает сам, когда данные поменялись. Это и есть рекомпозиция.

Шаблоны проектов

Android Studio создаёт проект из шаблона — важно не перепутать

■ ШАБЛОН 01

Empty Activity

Наш шаблон — Compose + Kotlin

Именно его вы выберете, когда будете создавать проект дома. Под капотом: Jetpack Compose, Kotlin DSL, Version Catalog — всё современное.

COMPOSE

KOTLIN

KOTLIN DSL

■ ШАБЛОН 02

Empty Views Activity

Старый мир — XML и View

НЕ нужен. В мастере два похожих названия — легко ошибиться. Обратите внимание при выборе.

XML

VIEW

LEGACY

■ НАСТРОЙКИ МАСТЕРА

Что задаём при создании

■ **Name** — название проекта (DemoApp)

■ **Language** — Kotlin (обязательно)

■ **Package name** — `com.example.demoapp`

■ **Minimum SDK** — API 24 (~97% устройств)

По умолчанию: Kotlin DSL + Version Catalog (`libs.versions.toml`)

FILE TREE • DEMOAPP

```
DemoApp/
├── settings.gradle.kts      # модули, репозитории
├── build.gradle.kts        # плагины (apply false)
├── gradle/
│   ├── libs.versions.toml  # Version Catalog
│   └── wrapper/            # фикс. версия Gradle
├── gradlew / gradlew.bat   # запуск сборки (Wrapper)
└── app/
    ├── build.gradle.kts    # compileSdk/minSdk, deps
    └── src/main/
        ├── AndroidManifest.xml
        ├── java/.../
        │   └── MainActivity.kt  # КОД
        └── res/                # строки, цвета, тема
```

Не пытайтесь запомнить — будете трогать руками всю неделю

■ ШАБЛОН EMPTY ACTIVITY

Структура проекта DemoApp

Что вы получите при создании проекта из шаблона

- 01 `settings.gradle.kts` — точка входа для сборщика: модули и репозитории.
- 02 `gradle/libs.versions.toml` — **Version Catalog**: все версии библиотек в одном файле.
- 03 `AndroidManifest.xml` — «паспорт» приложения: Activity, разрешения, метаданные.
- 04 `MainActivity.kt` — наш код, сегодня самый важный файл.

`./gradlew assembleDebug`

Сборка debug-APK через Gradle Wrapper

От Hello World к калькулятору

4 шага — живой код воркшопа. Template Empty Activity → рабочий калькулятор

01

STEP 01

Экран-дисплей

Сверху — большой `Text`, показывает текущее число. Это composable-функция, которые мы только что разобрали.

```
@Composable
fun Display(value: String) {
    Text(
        text = value,
        fontSize = 48.sp
    )
}
```

02

STEP 02

Состояние

Ключевая строчка: `var display by remember { mutableStateOf("0") }`. Когда данные меняются — Compose **сам перерисовывает** дисплей.

```
var display by remember {
    mutableStateOf("0")
}
// display = "7"
// → UI сам обновится
```

03

STEP 03

Клавиатура

`Row` и `Column` из кнопок. У каждой — `onClick`. Кнопка «7» делает `display = "7"`, кнопка «+» запоминает операцию.

```
Row {
    Button(onClick = {
        display = "7"
    }) {
        Text("7")
    }
}
```

04

STEP 04

Чистая логика

Вычисления в обычных функциях `compute` и `format`. Без UI — легко покрыть юнит-тестами. Деление на 0 → «Ошибка».

```
fun compute(
    a: Double,
    b: Double,
    op: Op
): Double = when(op) {
    ADD → a + b
    DIV → if (b ≠ 0.0)
        a / b
    else Double.NaN
    // ...
}
```

■ BUILD FLOW

Собираем APK и тестируем

Полный цикл: код → APK → установка на устройство

```
# 1. Сборка APK (в терминале)
./gradlew assembleDebug

# 2. Результат
app/build/outputs/apk/debug/
└─ app-debug.apk

# 3. Установка на телефон по USB
adb install app-debug.apk
```

АЛЬТЕРНАТИВА

В студии: **Build** → **Build App Bundle(s)/APK(s)** → **Build APK(s)**. Или запустить Run ► — установит на эмулятор автоматически.

НА ТЕЛЕФОНЕ

Включить «Параметры разработчика» → «Отладка по USB» → подключить кабель → **adb install app-debug.apk**.

SDK VERSIONS

3 параметра в build.gradle.kts

- **compileSdk = 37** — по какому API компилируем
- **targetSdk = 37** — на что заявляем совместимость
- **minSdk = 24** — минимум, ~97% устройств

Правило: $\text{minSdk} \leq \text{targetSdk} \leq \text{compileSdk}$

Play с 31.08.2026 требует $\text{targetSdk} \geq 36$. У нас 37 — ок.

APK VS AAB

APK — ставится напрямую. Наш рабочий формат для демо и проекта.

AAB (App Bundle) — формат публикации в Play. Play сам собирает APK под устройство (~30–60% выигрыш размера). Обязателен для новых приложений с 2021.

ДЛЯ КУРСА → APK

Рoadmap курса и ближайшие шаги

Куда идём — 8 разделов. Что сделать на этой неделе.

■ КУРС • 8 РАЗДЕЛОВ

01**Введение и инструменты**

Где мы сейчас

02**Kotlin: основы**

Язык, типы, ООП

03**Kotlin: продвинутый**

Coroutines, generics, IO

04**Compose: компоненты**

Базовый UI

05**Compose: состояние**

ViewModel, Flow

06**Навигация + архитектура**

Single-activity

07**Данные: Room, Retrofit**

Хранение и сеть

08**Сеть и тесты**

Финальный проект

■ ЗАДАНИЕ 01

Что сделать на этой неделе

- **Установить Android Studio** по инструкции в чате (`INSTALL_ANDROID_STUDIO.md`). Лучше прийти готовым.
- **Начать думать над темой** своего приложения — какая задача интересна, что хотелось бы сделать.

Дедлайн и критерии — в тексте задания в репозитории

■ МАТЕРИАЛЫ

Куда смотреть вперёд

- developer.android.com/guide/platform
- developer.android.com/develop/ui/compose
- kotlinlang.org/docs/android-overview.html
- developer.android.com/courses/android-basics-compose/course

Также: Habr (2025) — корутины для новичков